Aula 18 – Funções em linguagem C: Introdução

1. Objetivos de aprendizagem

- Compreender o conceito e a importância das funções em Linguagem C.
- Identificar os elementos que compõem uma função.
- Diferenciar parâmetros de argumentos.
- Reconhecer e implementar os quatro tipos de funções.
- Compreender o funcionamento de void e return.
- Identificar o escopo das variáveis locais.
- Criar programas simples utilizando funções para organizar e reutilizar códigos.

2. Relembrando...

Nas aulas anteriores, utilizamos funções prontas da Linguagem C, como printf(), scanf(), strlen() e strcmp(). Essas funções executam tarefas específicas e podem ser utilizadas sempre que necessário.

Mas será que podemos criar nossas próprias funções?

Imagine um programa que precisa calcular a soma de dois números em diferentes momentos. Sem funções, seria necessário repetir os comandos de cálculo. Com funções, podemos escrever a operação uma única vez e reutilizá-la.

Como podemos evitar a repetição de comandos e tornar nossos programas mais organizados?

3. O que são funções?

Funções são blocos de código reutilizáveis que executam uma tarefa específica. Elas permitem dividir um programa em partes menores, facilitando a compreensão, a manutenção e a reutilização de comandos.

Exemplo: sem utilizar uma função

```
#include <stdio.h>

int main(void) {
    int resultado1 = 5 + 3;
    printf("Resultado: %d\n", resultado1);

    int resultado2 = 10 + 20;
    printf("Resultado: %d\n", resultado2);

    return 0;
}
```

Observe que a operação de soma foi escrita duas vezes.

Exemplo: utilizando uma função

```
#include <stdio.h>

int soma(int n1, int n2) {
    return n1 + n2;
}

int main(void) {
    printf("Resultado: %d\n", soma(5, 3));
    printf("Resultado: %d\n", soma(10, 20));

    return 0;
}
```

Saída esperada:

Resultado: 8
Resultado: 30

Agora, a operação foi definida uma única vez e utilizada em diferentes momentos do programa.

Principais vantagens das funções

- Reutilização: Evitam repetir o mesmo código.
- Organização: Dividem programas maiores em partes menores.
- Legibilidade: Facilitam a compreensão do código.
- Manutenção: Permitem corrigir ou modificar uma operação em um único lugar.
- Testabilidade: Facilitam a verificação individual de cada operação.

4. Estrutura de uma função em C

Uma função pode ser representada pela seguinte estrutura:

```
tipo_retorno nome_funcao(parametros) {
  // Comandos executados pela funcao
  return valor;
}
```

O comando return valor; é utilizado quando a função precisa devolver um resultado. Funções com retorno void não devolvem um valor.

Modularização de programas

A modularização é uma técnica de programação que consiste em dividir um programa em partes menores e independentes, chamadas módulos, cada uma responsável por executar uma tarefa específica.

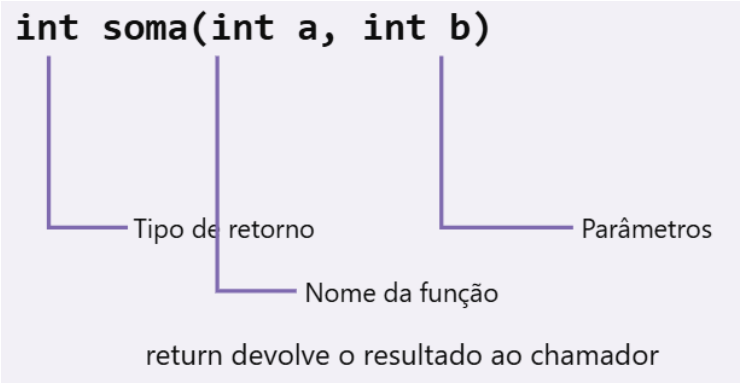
Na Linguagem C, uma das principais formas de aplicar a modularização é por meio da criação de funções. Essa organização facilita o desenvolvimento, a leitura, a manutenção e a reutilização do código.

Exemplo prático

Imagine um sistema que precisa cadastrar um aluno, calcular sua média e apresentar sua situação acadêmica. Sem modularização, todas essas operações poderiam ficar concentradas na função main().

Com modularização, podemos organizar o programa em funções específicas.

Anatomia de uma função



Elemento	Finalidade
Tipo de retorno	Define o tipo do resultado devolvido
Nome	Identifica a função
Parâmetros	Recebem os dados fornecidos
Corpo	Contém os comandos executados
`return`	Encerra a função e pode devolver um valor

5. Diferença entre parâmetros e argumentos

Embora estejam relacionados, parâmetros e argumentos não são a mesma coisa.

Parâmetros são as variáveis declaradas na definição da função para receber dados.

Argumentos são os valores ou expressões informados no momento da chamada.

```
#include <stdio.h>

int soma(int a, int b) {
    return a + b;
}

int main(void) {
    int resultado = soma(5, 3);
```

```
    printf("Resultado: %d\n", resultado);

    return 0;
}
```

Nesse exemplo:

- a e b são os parâmetros.
- 5 e 3 são os argumentos.
- soma(5, 3) é a chamada da função.

A função recebe os valores, realiza o cálculo e devolve o resultado.

6. Os quatro tipos de funções

As funções podem ser classificadas conforme a presença de parâmetros e de um valor de retorno.

Tipo	Recebe parâmetros?	Retorna valor?
1. Sem parâmetros e sem retorno	Não	Não
2. Com parâmetros e sem retorno	Sim	Não
3. Sem parâmetros e com retorno	Não	Sim
4. Com parâmetros e com retorno	Sim	Sim

Tipo 1 – Função sem parâmetros e sem retorno

Esse tipo de função executa uma tarefa, mas não recebe informações por parâmetros e não devolve valores.

Exemplo: exibir uma mensagem.

```
#include <stdio.h>

void mensagem(void) {
    printf("Bem-vindo ao sistema!\n");
}

int main(void) {
    mensagem();

    return 0;
}
```

Saída: *Bem-vindo ao sistema!*

A palavra void antes do nome da função indica que ela não retorna um valor. O void entre parênteses indica que ela não recebe parâmetros.

Tipo 2 – Função com parâmetros e sem retorno

Esse tipo de função recebe informações para realizar uma tarefa, mas não devolve um valor ao programa que realizou a chamada.

Exemplo: saudação personalizada.

```
#include <stdio.h>

void saudacao(char nome[]) {
    printf("Ola, %s!\n", nome);
}

int main(void) {
    saudacao("Ana");
    saudacao("Carlos");

    return 0;
}
```

Saída: *Ola, Ana! Ola, Carlos!*

A função recebe um nome e apresenta a mensagem correspondente.

Tipo 3 – Função sem parâmetros e com retorno

Esse tipo de função não recebe informações por parâmetros, mas devolve um resultado.

Exemplo: obter um número digitado pelo usuário.

```
#include <stdio.h>

int obterNumero(void) {
    int numero;

    printf("Digite um numero: ");
    scanf("%d", &numero);

    return numero;
}

int main(void) {
    int valor = obterNumero();

    printf("Numero informado: %d\n", valor);

    return 0;
}
```

Exemplo de execução:

Digite um numero: 15
Numero informado: 15

A função obterNumero() realiza a leitura e retorna o número informado. Neste exemplo introdutório, considera-se uma entrada inteira válida.

Tipo 4 – Função com parâmetros e com retorno

Esse tipo de função recebe informações, realiza um processamento e devolve um resultado.

Exemplo: calcular o dobro de um número.

```
#include <stdio.h>

int dobro(int numero) {
    return numero * 2;
}

int main(void) {
    int valor = 8;
    int resultado = dobro(valor);

    printf("Dobro: %d\n", resultado);

    return 0;
}
```

Saída:

Dobro: 16

A função recebe o número 8, calcula seu dobro e retorna 16.

7. Entendendo o fluxo de execução

Quando o programa encontra uma chamada de função, a execução é transferida para ela. Após a conclusão da função, o programa continua a partir do ponto seguinte à chamada.

```
#include <stdio.h>

void mensagem(void) {
    printf("Executando a funcao.\n");
}

int main(void) {
    printf("Inicio do programa.\n");

    mensagem();

    printf("Fim do programa.\n");
```

```
    return 0;
}
```

Saída:

Inicio do programa.
Executando a funcao.
Fim do programa.

Fluxo de execução de uma função



Pergunta para reflexão: Se a função mensagem() for chamada três vezes, quantas vezes sua mensagem será exibida?

8. A importância do comando return

O comando return encerra a execução de uma função. Quando a função possui um tipo de retorno diferente de void, ele também pode devolver um valor.

Exemplo: calcular o quadrado de um número.

```
#include <stdio.h>
```

```c
int calcularQuadrado(int numero) {
    return numero * numero;
}

int main(void) {
    int resultado = calcularQuadrado(6);

    printf("Quadrado: %d\n", resultado);

    return 0;
}
```

Saída: *Quadrado: 36*

Diferença entre printf() e return

`printf()`	`return`
Exibe informações na tela	Encerra a função e pode devolver um valor
É uma função da biblioteca de entrada e saída	É um comando da linguagem
Não serve para devolver o resultado ao chamador	Permite que o resultado seja utilizado pelo chamador

Por exemplo:

int total = calcularQuadrado(4) + 10;

Nesse caso, total receberá 26.

9. Passagem de parâmetros por valor

Na Linguagem C, os parâmetros de tipos simples, como int e float, recebem cópias dos valores dos argumentos. Isso significa que alterações feitas nesses parâmetros não modificam diretamente as variáveis originais.

```c
#include <stdio.h>

void dobrar(int n) {
    n = n * 2;
    printf("Dentro da funcao: %d\n", n);
}

int main(void) {
    int x = 5;

    dobrar(x);
```

```
    printf("No main: %d\n", x);

    return 0;
}
```

Saída:

Dentro da funcao: 10
No main: 5

A variável x continua com valor 5, pois a função modificou apenas o parâmetro local n.

Importante: Quando estudarmos ponteiros, veremos como uma função pode receber um endereço e modificar indiretamente uma variável original.

10. Variáveis locais e escopo

Variáveis declaradas dentro de uma função são chamadas de variáveis locais. Seu nome só pode ser utilizado dentro do bloco em que foram declaradas, respeitando as regras de escopo da linguagem.

```
#include <stdio.h>

void exemplo(void) {
    int numero = 10;

    printf("Valor: %d\n", numero);
}

int main(void) {
    exemplo();

    return 0;
}
```

A variável numero foi declarada dentro da função exemplo(). Se tentarmos utilizá-la diretamente em main(), ocorrerá um erro de compilação, pois seu nome não está disponível naquele escopo.

Exemplo de utilização incorreta

```
int main(void) {
    exemplo();

    printf("%d", numero); // Erro

    return 0;
}
```

Nesse trecho, considerando a função exemplo() apresentada anteriormente, numero não foi declarado no escopo de main().

11. Declaração, definição e chamada de funções

Em C, uma função precisa ser declarada antes de sua utilização. Quando a definição aparece depois de main(), podemos utilizar um protótipo.

```
#include <stdio.h>

// Declaracao (prototipo)
int multiplicar(int a, int b);

int main(void) {
    int resultado = multiplicar(4, 5);

    printf("Resultado: %d\n", resultado);

    return 0;
}

// Definicao
int multiplicar(int a, int b) {
   return a * b;
}
```

Nesse programa:

- Declaração: informa ao compilador a assinatura da função.
- Definição: contém os comandos que serão executados.
- Chamada: solicita a execução da função.

A saída será: Resultado: 20

Atividade para entrega

1º Questão: Uma empresa deseja criar uma calculadora simples para realizar soma, subtração, multiplicação e divisão.

Requisitos

1. Criar uma função exibirMenu() para apresentar as operações.
2. Criar as funções somar(), subtrair(), multiplicar() e dividir().
3. Cada função matemática deverá receber dois números e retornar o resultado.
4. Utilizar switch para selecionar a operação.
5. Impedir a divisão por zero.
6. Permitir novas operações até que o usuário escolha sair.

Exemplo de saída:

```
===== CALCULADORA =====

1 - Somar
2 - Subtrair
3 - Multiplicar
4 - Dividir
0 - Sair

Escolha uma opcao: 1

Primeiro numero: 10
Segundo numero: 5

Resultado: 15.00
```

2º Questão: Uma loja precisa comparar os preços de dois produtos.

Requisitos:

1. Solicitar dois preços.
2. Criar uma função maiorPreco() que receba os valores e retorne o maior.
3. Apresentar o resultado no programa principal.
4. Informar quando os preços forem iguais.

Exemplo de saída:

```
===== COMPARADOR DE PRECOS =====

Preco 1: 150.00
Preco 2: 200.00

Maior preco: R$ 200.00
```

3º Questão: Um sistema precisa identificar se um número é par ou ímpar.

Requisitos:

1. Solicitar um número inteiro.
2. Criar uma função verificarPar().
3. Retornar 1 para números pares e 0 para ímpares.
4. Apresentar o resultado.

Exemplo de saída:

```
===== VERIFICADOR DE NUMEROS =====

Digite um numero: 8

O numero 8 e par.
```

4º Questão: uma empresa deseja calcular a quantidade de produtos após duplicar seu estoque.

Requisitos:

1. Solicitar a quantidade inicial.
2. Criar uma função dobro() que receba a quantidade.
3. Retornar o dobro.
4. Exibir a quantidade inicial e final.

```
===== CONTROLE DE ESTOQUE =====

Quantidade inicial: 15

Quantidade apos duplicacao: 30
```

5º Questão: um sistema precisa apresentar relatórios com separadores visuais.

Requisitos:

1. Criar uma função exibirLinha() sem parâmetros e sem retorno.
2. Imprimir dez asteriscos.
3. Chamar a função antes e depois do título de um relatório.

```
**********
RELATORIO DE ESTOQUE
**********
```

6º Questão: desenvolver um programa que calcule o fatorial de um número inteiro não negativo.

Requisitos:

1. Solicitar um número entre 0 e 12.
2. Criar uma função calcularFatorial().
3. Utilizar for ou while para realizar o cálculo.
4. Retornar o resultado.
5. Apresentar o fatorial calculado.

```
===== CALCULADORA DE FATORIAL =====

Digite um numero: 5

Fatorial de 5 = 120
```

Orientações para entrega no GitHub

Os alunos deverão organizar os exercícios no repositório laboratorio-programacao-c, criando a pasta aula-18-funcoes/.

```
laboratorio-programacao-c/
└── aula-18-funcoes/
    ├── calculadora.c
    ├── comparador_precos.c
    ├── verificador_numeros.c
    ├── controle_estoque.c
    ├── formatacao_relatorios.c
    └── calculadora_fatorial.c
```

Atividade de revisão:

1. Qual função não possui parâmetros nem retorno?
2. Quais funções recebem dois parâmetros?
3. Qual é a finalidade do return nas funções matemáticas?
4. Por que o programa utiliza switch?
5. Qual é a finalidade da estrutura do...while?
6. Por que a divisão por zero precisa ser verificada?

Estudos complementares

Para aprofundar os conhecimentos, seguem dois materiais que já constavam na bibliografia utilizada no semestre passado:

- Programação Prática – Funções em C
- IFSC – Material didático sobre funções em C

Durante a leitura, observe como as funções são declaradas, definidas e chamadas, além das diferenças entre parâmetros e argumentos.